16

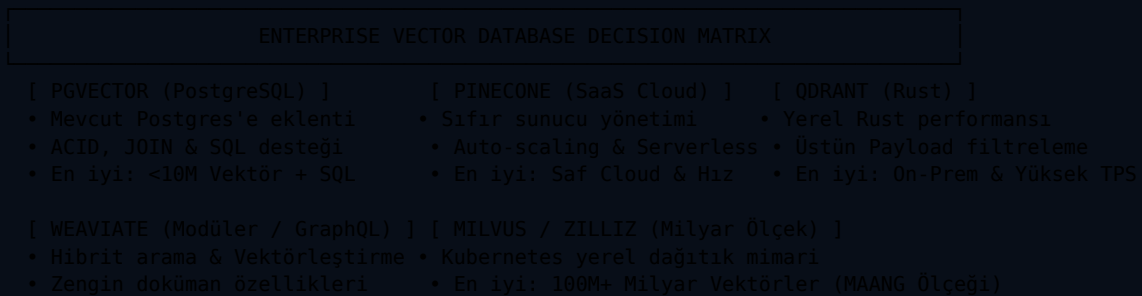
BÖLÜM: VEKTÖRLER, EMBEDDING'LER VE VEKTÖR VERITABANLARI | MODULE: VECTORS, EMBEDDINGS & VECTOR DATABASE ARCHITECTURE

Vektör Veritabanı Seçimi: pgvector vs Pinecone vs Qdrant vs Weaviate vs Milvus

Vector Database Evaluation: pgvector vs Pinecone vs Qdrant vs Weaviate vs Milvus

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Vektör veritabanı seçmek araba seçmek gibidir: Eğer zaten garajınızda sağlam bir aile arabanız varsa (PostgreSQL), arkasına bir çeki demiri takıp (pgvector eklentisi) 100.000 dokümanınızı rahatça taşıyabilirsiniz; ekstra masrafa gerek yoktur! Ama bir lojistik şirketiyseniz ve her gün 500 milyon koli taşıyacaksanız, özel bir tır filosu (Milvus veya Pinecone) kiralamanız gerekir. MAANG mülakatında 'Hemen Pinecone kuralım' derseniz elenirsiniz; 'Önce veri hacmimiz kaç? Mevcut ilişkisel verimizle JOIN yapacak mıyız?' diye sormanız gerekir!

🇬🇧 ENGLISH (ELI5)

Selecting a vector store is like choosing transportation: If you already run a reliable enterprise fleet of delivery vans (PostgreSQL), installing a trailer hitch (the `pgvector` extension) handles 500,000 embeddings with ACID compliance and zero architectural overhead. But if you are managing a global airline logistics network with 500 million packages, you deploy purpose-built cargo jets (Pinecone or Milvus). In MAANG interviews, never blindly default to specialized vector SaaS without evaluating operational complexity!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Karar Matrisi Kriterleri: 1) pgvector: Eğer doküman sayısı 5 milyonun altındaysa ve kullanıcı tablosu, faturalar veya yetkilendirme (RBAC) ile SQL JOIN gerekiyorsa pgvector tartışmasız en iyi seçimdir. Ayrı bir veritabanı yönetme ve veri senkronizasyonu (CDC) derdini bitirir. 2) Pinecone: Tamamen yönetilen (Serverless) SaaS'tır. Altyapı yönetmek istemeyen bulut ekipleri için idealdir ancak veriler şirket dışına çıkar. 3) Qdrant: Rust ile yazılmıştır, bellek verimliliği mükemmeldir ve karmaşık metadata filtrelemelerinde (Payload Indexing) rakipsizdir. 4) Milvus: 100 milyondan fazla vektörü Kubernetes üzerinde sharding yaparak dağıtık yönetmek için tasarlanmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architectural Trade-Off Matrix: 1) pgvector: Unbeatable for systems <5M vectors where transactional ACID guarantees and relational SQL JOINs with user/permission tables dominate. Eliminates dual-write consistency issues. 2) Pinecone: Fully managed Serverless vector SaaS; minimizes operational toil but introduces third-party data sovereignty and egress concerns. 3) Qdrant: Bare-metal Rust engine delivering peerless memory efficiency and hardware-accelerated payload filtering. 4) Milvus: Distributed microservice architecture built for billion-scale embeddings across Kubernetes clusters.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25: Vektör Veritabanı Seçimini Belirleyen Karar Motoru
public class VectorStoreDecisionEngine {
    public enum VectorStoreChoice { PGVECTOR, PINECONE_SAAS, QDRANT_RUST, MILVUS_DISTRIBUTED }

    public static VectorStoreChoice evaluate(long totalVectors, boolean requiresRelationalJoin,
boolean isCloudSaaSPreferred) {
        if (totalVectors < 5_000_000 && requiresRelationalJoin) {
            return VectorStoreChoice.PGVECTOR; // Postgres ile devam et, mimariyi sade tut!
        }
        if (isCloudSaaSPreferred && totalVectors < 50_000_000) {
            return VectorStoreChoice.PINECONE_SAAS;
        }
        if (totalVectors > 100_000_000) {
            return VectorStoreChoice.MILVUS_DISTRIBUTED; // Milyar ölçekte dağıtık mimari
        }
        return VectorStoreChoice.QDRANT_RUST;
    }
}

void main() {
    var choice = VectorStoreDecisionEngine.evaluate(250_000, true, false);
    IO.println("Önerilen Vektör Veritabanı: " + choice); // PGVECTOR
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Uyarısı: Ayrı bir vektör veritabanı (ör. Pinecone) kullandığınızda, ana veritabanınızdaki (MySQL/Postgres) veriler güncellendiğinde veya silindiğinde iki veritabanı arasında tutarsızlık (Data Drift) oluşur. Bu senkronizasyonu sağlamak için Debezium CDC veya Outbox Pattern anlatmanız beklenir!

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Dual-Write consistency trap: Introducing a standalone vector store beside your relational database creates dual-write split-brain risks. Deleting a record in Postgres without an event-driven sync (CDC / Kafka) leaves orphaned embeddings in Pinecone.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• 5 Milyon vektörün altındaki çoğu senaryoda PostgreSQL + pgvector en pratik ve güçlü çözümdür.

• İlişkisel verilerle (Kullanıcı, İzinler, Roller) vektörleri aynı tabloda JOIN edebilmek büyük nimettir.

• Yüz milyonlarca vektör için Milvus veya Qdrant gibi özelleşmiş motorlara geçilmelidir.

🎯 Key Takeaways for the Interview

• For datasets under 5M vectors, PostgreSQL + pgvector eliminates dual-write architectural overhead.

• Co-locating relational permissions and vector embeddings prevents security filter bypasses.

• Billions of embeddings justify dedicated distributed vector engines like Milvus or Qdrant.